

# pypresso

what it computes, and how to ask for it

Plane-wave DFT in Python and JAX, validated against Quantum ESPRESSO

August 26, 2026

## Abstract

User documentation for the feature set: every capability, the equation behind it, a snippet that runs it, what it was validated against, and *what it refuses*. Two conventions run through the whole document.

**Everything is validated against Quantum ESPRESSO on the same input**, and the agreement is quoted per feature rather than claimed in general. Where a number is missing it is because the comparison does not exist, and that is said rather than hidden.

**Anything not implemented is refused by name**, with an error saying what is missing — never silently approximated. The “Refuses” notes are as much of the documentation as the features: they are the promise that a run which starts is a run whose physics is there.

---

**One equation is written down and the rest are derived from it.** Almost everything below is a derivative of the total energy, taken by the compiler rather than by hand:

$$E[\{\psi\}, \tau, \varepsilon] = T_s + E_H + E_{xc} + E_{\text{loc}} + E_{\text{nl}} + E_{\text{Ewald}}$$

| quantity                               | what it is                                                    | how it is obtained                        |
|----------------------------------------|---------------------------------------------------------------|-------------------------------------------|
| $v_{xc}(\mathbf{r})$                   | $\delta E_{xc}/\delta n$                                      | jax.grad of the energy density            |
| $\mathbf{F}_I$                         | $-\partial E/\partial \boldsymbol{\tau}_I$                    | jax.grad at frozen $\psi$                 |
| $\sigma_{ij}$                          | $-\Omega^{-1}\partial E/\partial \varepsilon_{ij}$            | jax.grad along a strain                   |
| $C_{I\alpha, J\beta}$                  | $\partial^2 E/\partial \tau_{I\alpha} \partial \tau_{J\beta}$ | jvp of the force                          |
| $Z_{I, \alpha\beta}^*$                 | $\partial F_{I\alpha}/\partial \mathcal{E}_\beta$             | jvp of the force along the field response |
| $\partial \epsilon_{ij}/\partial \tau$ | Raman                                                         | jvp of the second-order energy            |

The “frozen  $\psi$ ” is what makes this exact rather than approximate: at self-consistency the energy is stationary with respect to the wavefunctions, so the terms from  $\partial\psi/\partial\lambda$  vanish. That is the Hellmann-Feynman argument, and taking it one order up is the  $2n+1$  theorem, which is why a *third* derivative needs only first-order wavefunctions.

# Contents

|           |                                                     |           |
|-----------|-----------------------------------------------------|-----------|
| <b>1</b>  | <b>Input and invocation</b>                         | <b>3</b>  |
| <b>2</b>  | <b>Ground state</b>                                 | <b>4</b>  |
| <b>3</b>  | <b>Bands, densities of states, projections</b>      | <b>4</b>  |
| <b>4</b>  | <b>Pseudopotentials and functionals</b>             | <b>5</b>  |
| <b>5</b>  | <b>Spin, magnetism and spin-orbit</b>               | <b>6</b>  |
| <b>6</b>  | <b>Forces, stress and relaxation</b>                | <b>6</b>  |
| <b>7</b>  | <b>Response: dielectric, phonons, Raman</b>         | <b>7</b>  |
| <b>8</b>  | <b>Topological invariants</b>                       | <b>9</b>  |
| <b>9</b>  | <b>Things with no <code>pw.x</code> counterpart</b> | <b>9</b>  |
| <b>10</b> | <b>Performance: dials, memory and GPUs</b>          | <b>10</b> |
| <b>11</b> | <b>Accuracy summary</b>                             | <b>11</b> |
|           | <b>Where to go next</b>                             | <b>11</b> |

# 1 Input and invocation

pypresso reads **pw.x input files**. The same file that runs in Quantum ESPRESSO runs here, which is what makes every comparison in this document possible.

```
from pypresso.io.pwin import read_pw_input
from pypresso.pseudo import read_upf
from pypresso.scf.driver import Calculation, run_scf
from pypresso.system import build_system

system = build_system(read_pw_input("scf.in"))
pseudos = tuple(read_upf(f"pseudo/{s.pseudo_file}") for s in system.structure.species)
result = run_scf(system, pseudos, conv_thr=1.0e-8)

print(result.total_energy, "Ry in", result.iterations, "iterations")
```

conv\_thr means what it means in a pw.x input: it is compared against QE's dr2, the Hartree energy of the density residual. A command line covers the common workflows:

```
python3 -m pypresso.cli inspect <qe-output> # summarise what the parser reads
python3 -m pypresso.cli dos <input> # density of states
python3 -m pypresso.cli pdos <input> # projected DOS
python3 -m pypresso.cli relax <input> # structural relaxation
python3 -m pypresso.cli stress <input> # stress tensor
python3 -m pypresso.cli spiral <input> # spin-spiral scan
```

Standard pw.x variables — ecutwfc, ecutrho, ibrav, nbnd, occupations, degauss and the rest — mean what they mean there and are not re-documented here. A handful of knobs are worth knowing because they are specific to this code or to a feature below:

|                        |                                                           |
|------------------------|-----------------------------------------------------------|
| mbj_c                  | fixes Tran-Blaha's $c$ instead of taking the cell average |
| london_s6, london_rcut | Grimme D2's scaling and real-space cutoff                 |
| cell_dofree            | which cell degrees of freedom a vc-relax may move         |
| starting_ns_eigenvalue | seeds the DFT+U occupation matrix                         |
| spiral_q               | the spin-spiral wavevector, in lattice coordinates        |
| LOCAL_MAGNETIC_FIELDS  | a card with no pw.x counterpart                           |

**Units are Rydberg atomic units throughout** (Ry, bohr), matching QE. The only layer that speaks anything else is io/: Hubbard  $U$  in the HUBBARD card is in eV and is converted at the boundary, as pw.x does it.

**Refuses.** K\_POINTS gamma selects the half-sphere storage of the gamma-point trick, which is generated but not consumed. Such a run is substituted by an explicit  $k = 0$  on the full sphere — the same physics at twice the storage — and it says so.

## 2 Ground state

The Kohn-Sham problem, generalised because ultrasoft and PAW make the overlap non-trivial:

$$\begin{aligned}\hat{H}|\psi_{n\mathbf{k}}\rangle &= \epsilon_{n\mathbf{k}} \hat{S}|\psi_{n\mathbf{k}}\rangle, & \hat{S} &= 1 + \sum_{I,ij} q_{ij} |\beta_i^I\rangle\langle\beta_j^I| \\ \hat{H} &= -\nabla^2 + v_{\text{loc}}(\mathbf{r}) + v_{\text{H}}[n] + v_{xc}[n] & & + \sum_{I,ij} D_{ij}^I |\beta_i^I\rangle\langle\beta_j^I|\end{aligned}$$

with  $n(\mathbf{r}) = \sum_{n\mathbf{k}} w_{\mathbf{k}} f_{n\mathbf{k}} (|\psi_{n\mathbf{k}}|^2 + \sum_{ij} Q_{ij}^I \langle\psi|\beta_i\rangle\langle\beta_j|\psi\rangle)$ , the augmentation term being what ultrasoft adds.  $D_{ij}^I$  is rebuilt from the potential every iteration, which is why the SCF is a fixed point in  $(n, D)$  and not in  $n$  alone. Convergence is measured as QE measures it — the Hartree energy of the residual:

$$dr^2 = \iint \frac{\delta n(\mathbf{r}) \delta n(\mathbf{r}')}{|\mathbf{r} - \mathbf{r}'|} d\mathbf{r} d\mathbf{r}' < \text{conv\_thr}$$

```
result = run_scf(system, pseudos, conv_thr=1.0e-10, mixing_mode="anderson")
print(result.total_energy, result.energy_terms) # QE's own term-by-term split
print(result.fermi_energy, result.homo, result.lumo)
```

**Metals** work with every smearing QE has (gaussian, marzari-vanderbilt, methfessel-paxton, fermi-dirac) and with the tetrahedron family — both linear and Blochl-corrected — usable as an occupation scheme *inside* the SCF, not only for a density of states.

**Continuation across a change of spin regime.** Give `run_scf` a previous result as `starting_from` — `System.with_spin` underneath — and it promotes a converged state into a target's variables: non-magnetic into collinear, collinear into noncollinear, spin-orbit switched on. It reaches the *same* solution as a fresh run ( $\leq 4\text{e-}8$  Ry on six cases) in as little as one iteration. The magnetization is **seeded** from the target's `starting_magnetization` when the source has none, because nothing in the SCF breaks spin symmetry on its own and an unseeded promotion would converge back to the unpolarized solution and report success.

**Convergence.** `mixing_mode` selects `anderson` (default), `linear`, `broyden`, or Kerker/Thomas-Fermi preconditioning (TF); `scf_solver` offers a Newton-Krylov alternative that is slower but reaches solutions the mixer cannot hold. `max_iterations` defaults to 100 — **a hard SCF may need more**, and hitting the cap is reported as not converged rather than as an answer.

## 3 Bands, densities of states, projections

A band structure is one diagonalisation per  $\mathbf{k}$  on a fixed density — `run_bands`, or `run_nscf` for a non-self-consistent run on a grid rather than a path. A density of states is

$$g(E) = \sum_{n\mathbf{k}} w_{\mathbf{k}} \delta(E - \epsilon_{n\mathbf{k}})$$

with  $\delta$  replaced by a smearing function or evaluated by tetrahedron interpolation. The projected DOS weights each term by  $|\langle\phi_\mu|\hat{S}|\psi_{n\mathbf{k}}\rangle|^2$ , and the spilling parameter is what is *not* captured,  $\sum_{n\mathbf{k}} w_{\mathbf{k}} f_{n\mathbf{k}} (1 - \sum_\mu |\langle\phi_\mu|\hat{S}|\psi_{n\mathbf{k}}\rangle|^2)$ .

```

from pypresso.workflows.bands import run_bands
from pypresso.workflows.dos import run_dos
from pypresso.workflows.pdos import run_pdos

bands = run_bands(system, pseudos, scf.density, kpoints=path)  # a k-path

dos, _ = run_dos(system, pseudos, scf.density, grid=(12, 12, 12),
                 scheme="tetrahedra_opt")  # -> (DensityOfStates, NSCFResult)

pdos, _ = run_pdos(system, pseudos, scf)  # -> (ProjectedDOS, NSCFResult)
print(pdos.channels[0])  # resolved by atom, l and m
print(pdos.charges.charges, pdos.charges.spilling)

```

The DOS schemes live behind a name registry (DOS\_SCHEMES), and the projected DOS feeds *the same* registry as a per-band weight, so a PDOS and a DOS are the same integration. Projections are  $\langle \phi | \hat{S} | \psi \rangle$  on Löwdin-orthogonalised pseudo-atomic orbitals. Validated against a purpose-built projwfc.x on seven cases: **6.9e-4** on a projection, **4.7e-5** on a Löwdin charge, and the spilling parameter to all four decimals it prints.

## 4 Pseudopotentials and functionals

**Kinds:** norm-conserving, **ultrasoft** and **PAW** — the two-grid split, the augmentation charge, the overlap operator, self-consistent  $D_{ij}$ , and PAW’s one-centre terms including its radial Poisson solve and spherical quadrature. UPF v2 is read.

**Functionals:** LDA (PZ), **PBE**, **revPBE**, **PBEsol**, on the plane-wave grid and on the PAW spheres. The functional comes from the pseudopotential headers unless input\_dft overrides it. Only the *energy* is written down;  $v_{xc}$ , and a GGA’s  $v_1$  and  $v_2$ , come from jax.grad of it.

**Meta-GGA, potential-only branch:** input\_dft = 'tb09' (Tran-Blaha) and 'bj06' (Becke-Johnson). These invert the rule above — they *are* potentials, there is no energy — so the consequences are enforced rather than documented: run\_scf warns that its total is not the value of any functional it minimised, and forces, stress, phonons and response all refuse. Silicon’s gap goes from LDA’s 0.49 eV to **1.13 eV** against an experimental 1.17.

**Van der Waals:** Grimme **D2** (vdw\_corr = 'grimme-d2'). Bilayer graphene matches pw.x to **3.1e-9 Ry** and binds at 3.23 Å where PBE alone has no minimum.

**Refuses.** UPF v1; an unimplemented functional (rather than silently substituting LDA); energy-carrying meta-GGAs (TPSS, SCAN, M06L), whose potential has a  $\partial E / \partial \tau$  piece acting on the wavefunction that is not written; plain *ultrasoft* under a meta-GGA, which has no partial waves to reconstruct  $\tau$  from inside the sphere where PAW does; EXX; and **D3, Tkatchenko-Scheffler, MBD and XDM** — where pw.x warns and silently runs with no correction at all.

## 5 Spin, magnetism and spin-orbit

| regime       | how to ask          | note                                            |
|--------------|---------------------|-------------------------------------------------|
| unpolarised  | default             |                                                 |
| collinear    | nspin = 2           | one Fermi level, or two under tot_magnetization |
| noncollinear | noncolin = .true.   | magnetization is a vector                       |
| spin-orbit   | + lspinorb = .true. | j-resolved projectors; NC, US and PAW           |

**Three spin numbers are kept apart**, and `System` exposes all three: `nspin` says which regime is in force, `npol` is the number of spinor components of a *wavefunction*, and `nspin_mag` the number of components of a *density*. They coincide at 1 and 2 and come apart at 4, where `npol` = 2 but `nspin_mag` is **one** for a nonmagnetic spin-orbit run — which is why such a run costs about what a doubled unpolarized one costs.

**Magnetic fields and constraints:** a uniform field over the cell, a field inside one atom’s sphere (through a `LOCAL_MAGNETIC_FIELDS` card that `pw.x` has no counterpart for), Elk’s `reducebf`, and all four of QE’s `constrained_magnetization` schemes. **The field’s energy is not part of the reported total** — QE prints `etcon` and never adds it, and Elk excludes its external field’s energy by the same convention.

**DFT+U:** QE’s `lda_plus_u_kind` = 0 (Dudarev) with the  $J_0/\beta$  extension, on atomic, ortho-atomic and norm-atomic projectors, read from the `HUBBARD` card. Forces come from differentiating through projectors that move with the atoms.

**Refuses.** The full (Liechtenstein) DFT+U formulation, intersite  $V$ , background channels, the orbital-resolved variant, the `wf/pseudo` projector sets, and noncollinear `ns_nc`; and PAW + GGA + a noncollinear magnetization together.

## 6 Forces, stress and relaxation

$$\mathbf{F}_I = - \left. \frac{\partial E}{\partial \boldsymbol{\tau}_I} \right|_{\psi \text{ fixed}}, \quad \sigma_{ij} = - \left. \frac{1}{\Omega} \frac{\partial E}{\partial \varepsilon_{ij}} \right|_{\psi \text{ fixed}}$$

Both are one gradient. With ultrasoft the orthonormality constraint  $\langle \psi_n | \hat{S} | \psi_m \rangle = \delta_{nm}$  is carried explicitly, so its multiplier term — QE’s Pulay contribution — falls out of the same derivative rather than being added afterwards. A variable-cell relaxation minimises the **enthalpy**, which is why a relaxed crystal carries the applied pressure rather than having zero stress:

$$H = E + P \Omega, \quad \frac{\partial H}{\partial h} = \Omega (P \mathbf{1} - \sigma) h^{-T}$$

```

from pypresso.forces import compute_forces
from pypresso.stress import compute_stress
from pypresso.workflows.relax import run_relax
from pypresso.workflows.vc_relax import run_vc_relax

forces = compute_forces(calculation, result) # method=... for QE's own terms
print(forces.forces) # (nat, 3) in Ry/bohr

stress = compute_stress(calculation, result, terms=True)
print(stress.pressure) # kbar

relaxed = run_relax(system, pseudos, forc_conv_thr=1.0e-4)

```

```
cell = run_vc_relax(system, pseudos, press=500.0) # kbar
print(cell.pulay_error) # the frozen-basis error, reported not hidden
```

QE's `force_lc`, `force_cc`, `force_ew`, `force_us`, `addusforce` and `force_corr` are transcribed too, behind the same registry — **as a cross-check, not as the implementation**. The two share no machinery, which is what makes disagreement informative; it is how the augmentation force's sign and a missing gradient correction were found.

## 7 Response: dielectric, phonons, Raman

The **velocity operator** comes first and everything else uses it: it is one jvp of  $\hat{H}(\mathbf{k})$  at a frozen sphere, because  $\partial \hat{H} / \partial k_\alpha = i[\hat{H}, r_\alpha]$  in the periodic gauge. `band_velocities` exposes it, and because the overlap carries a velocity too, a band velocity is  $\langle \psi | \partial_k \hat{H} - \epsilon \partial_k \hat{S} | \psi \rangle$ .

Every first-order quantity then comes from the Sternheimer equation — the response of an occupied orbital to a perturbation, projected outside the occupied manifold so that no empty states and no energy denominators appear:

$$(\hat{H} - \epsilon_n \hat{S} + \alpha \hat{Q}) |\Delta \psi_n\rangle = -\hat{P}_c^+ \Delta V |\psi_n\rangle$$

It is solved self-consistently, because  $\Delta V$  contains the response of the potential to the response of the density,

$$\Delta V_{\text{scf}} = \Delta V_{\text{ext}} + \int K(\mathbf{r}, \mathbf{r}') \Delta n(\mathbf{r}') d\mathbf{r}'$$

and the kernel  $K$  is one jvp of `v_of_rho` rather than a second implementation. From its solutions:

$$\epsilon_{\alpha\beta}^\infty = \delta_{\alpha\beta} + \frac{8\pi}{\Omega} \sum_{\mathbf{n}\mathbf{k}} w_{\mathbf{k}} \langle \Delta_\alpha \psi_n | \partial_\beta \psi_n \rangle, \quad Z_{I,\alpha\beta}^* = \frac{\partial F_{I\alpha}}{\partial \mathcal{E}_\beta}$$

$$C_{I\alpha,J\beta} = \frac{\partial^2 E}{\partial \tau_{I\alpha} \partial \tau_{J\beta}}, \quad \omega^2 \mathbf{e} = \frac{1}{\sqrt{M_I M_J}} C \mathbf{e}, \quad \frac{\partial \epsilon_{ij}}{\partial \tau_{I\gamma}} \text{ (Raman)}$$

The Raman tensor is the second-order energy differentiated once more along a displacement, at *frozen first-order* wavefunctions — the  $2n+1$  theorem, which is why a third derivative costs one more jvp and not a new solve.

```
from pypresso.response import (dielectric_tensor, dynamical_matrix,
                               raman_tensors, vibrational_spectrum)

eps = dielectric_tensor(calculation, scf.wavefunctions, scf.eigenvalues,
                        scf.density)
print(eps.epsilon) # 3x3, cartesian

ph = dynamical_matrix(calculation, scf.wavefunctions, scf.eigenvalues,
                       scf.density, scf.becsum)
print(ph.frequencies) # cm^-1

raman = raman_tensors(calculation, scf, born_charges=True, keep_internals=True)
spectrum = vibrational_spectrum(calculation, scf) # per-mode Raman and IR
print(spectrum.raman_activity, spectrum.depolarisation)
```

born\_effective\_charges gives  $Z^*$  on its own, and mode\_activities is the bare assembly — a transcription of dynmat.x’s RamanIR, kept a pure function of arrays so it is testable without a self-consistent run.

Ultrasoft and PAW work for the dielectric constant ( $\leq 1.2e-4$  against ph.x on four cases); metals work for the response and the dynamical matrix; and a symmetry-reduced wedge works as of the rank- $n$  symmetriser.

**A trap worth knowing:** a response on a reduced  $k$ -set is a *polar vector field* and must be symmetrised as one. Running the whole grid instead is sound only if the grid is closed under the point group — a **shifted** Monkhorst-Pack grid is not.

**Refuses.** Born charges for PAW (int3\_paw against becsumort is missing, and without it the expression is wrong in sign as well as size);  $\chi^{(2)}$  and the electro-optic tensor (the second-order source term has nothing to build the  $\langle u_i | r_k | u_j \rangle$  piece from — and it is **42%** of the answer, measured); the non-analytic LO-TO term; phonons at  $q \neq 0$ ; the dynamical matrix of an ultrasoft dataset; any response under a potential-only meta-GGA; and a shifted grid with a reduced  $k$ -set.

## Elastic constants, electrostriction and the elasto-optic tensor

The same machinery in the *strain* coordinate rather than the atomic one. strain\_response solves the first-order problem for a strain — Abinit’s metric-tensor formulation, obtained for nothing here because the cell was already written in reduced coordinates — and two things follow from it:

$$C_{ijkl} = \frac{\partial \sigma_{ij}}{\partial \varepsilon_{kl}}, \quad \frac{\partial \chi_{ij}}{\partial \varepsilon_{kl}} \text{ (elasto-optic)}$$

The second is a **third** derivative of the energy, and it comes from one jvp of the second-order energy at frozen first-order wavefunctions — the  $2n+1$  theorem again. Through the thermodynamic identity of Tanner, Bousquet and Janolin it gives the four electrostriction tensors  $m$ ,  $q$ ,  $M$  and  $Q$ .

```
from pypresso.response import elastic_constants, electrostriction

es = electrostriction(calculation, scf)      # solves the strain response itself
print(es.m, es.q, es.M, es.Q)              # the four electrostriction tensors
print(es.photoelastic)                     # the elasto-optic tensor
print(es.elastic.voigt)                    # C_ij in Voigt notation, GPa

# or the elastic constants alone, from a strain response you already have
C = elastic_constants(calculation, scf.wavefunctions, scf.eigenvalues,
                      scf.density, es.strain)
print(C.tensor.shape, C.voigt)
```

Converged silicon gives  $C_{11} = 198.5$  GPa and  $C_{12} = 68.9$ , against a measured 165.7 and 63.9; the three independent components of  $\partial \epsilon / \partial \varepsilon$  match a central difference over re-converged strained cells to **2e-4**, which is that difference’s own floor, and the rank-4 tensor is cubic to 3e-14 with nothing imposing it.

**Refuses.** Both are norm-conserving, nspin = 1, insulators and **clamped-ion**. A diverged first-order solution is refused rather than consumed in silence (require\_converged\_responses) — at QE’s alpha\_mix = 0.7 the strain response of a *slab* di-



verges, and consuming it gave a  $C_{ijkl}$  that was not even symmetric under  $C_{ijkl} = C_{klij}$ . The elastic constants additionally need the whole  $k$ -grid rather than a wedge, because their functional builds its own density and symmetrises it as a scalar inside the chain rule.

## 8 Topological invariants

Everything is built from one primitive — the overlap of occupied manifolds at neighbouring  $k$ -points,

$$M_{mn}^{(\mathbf{b})} = \langle u_{m\mathbf{k}} | \hat{S} | u_{n,\mathbf{k}+\mathbf{b}} \rangle$$

because a determinant of overlaps is blind to the unitary mixing a degenerate eigensolver leaves. The Berry curvature is the phase of a plaquette of them and the Chern number their lattice sum, which is an **exact integer on any mesh**:

$$F_{12}(\mathbf{k}) = \ln [U_1(\mathbf{k})U_2(\mathbf{k}+\hat{1})U_1(\mathbf{k}+\hat{2})^{-1}U_2(\mathbf{k})^{-1}], \quad C = \frac{1}{2\pi i} \sum_{\mathbf{k}} F_{12}(\mathbf{k})$$

$Z_2$  has two independent routes — the flow of Wannier charge centres, and, when there is an inversion centre, the parity product over the eight TRIM points:

$$(-1)^\nu = \prod_{i=1}^8 \prod_{n \text{ occ}}^{\text{Kramers}} \xi_{2n}(\Gamma_i)$$

```
from pypresso.workflows.topology import (run_z2, run_z2_3d,
                                          run_berry_curvature)

z2 = run_z2(system, pseudos, scf.density, axis=2) # Wilson loop by default
print(z2.z2, z2.gap_step) # read gap_step before believing the integer

chern = run_berry_curvature(system, pseudos, scf.density, shape=(12, 12))
print(chern) # an exact integer on any mesh

nu = run_z2_3d(system, pseudos, scf.density) # the four 3D indices
print(nu) # (nu0; nu1 nu2 nu3)
```

**Running both  $Z_2$  routes is the check.** Where they disagree the parity one is the answer, because it has no mesh and the Wilson route does. **Two things bite in a plane-wave code and both are silent:** neighbouring  $k$ -points do not share a  $G$ -sphere, so coefficients are aligned by Miller index; and the wrap at the zone edge is a *shift* of that index, without which the Chern number comes out smooth and non-integer.

## 9 Things with no `pw.x` counterpart

**Spin spirals** by the generalized Bloch theorem: a spiral of wavevector  $\mathbf{q}$  is a spinor whose two components live on *different* plane-wave spheres,

$$\psi_{\mathbf{k}}(\mathbf{r}) = \begin{pmatrix} e^{i(\mathbf{k}-\mathbf{q}/2)\cdot\mathbf{r}} u^\uparrow(\mathbf{r}) \\ e^{i(\mathbf{k}+\mathbf{q}/2)\cdot\mathbf{r}} u^\downarrow(\mathbf{r}) \end{pmatrix}$$

so in the rotated frame the density is lattice periodic and the SCF, the functional and the mixer are untouched. Only two terms carry  $\mathbf{q} = |\mathbf{k} \mp \mathbf{q}/2 + \mathbf{G}|^2$  and  $v_{\text{nl}}$  — and  $dE/d\mathbf{q}$  at frozen coefficients is `jax.grad` of them.

```
from pypresso.workflows.spiral import (run_spiral_scan, relax_spiral_q,
                                       heisenberg_exchange)

scan = run_spiral_scan(system, pseudos, wavevectors) # E(q)
best = relax_spiral_q(system, pseudos) # BFGS on the reciprocal metric
print(best.wavevector, best.scf.total_energy)

J = heisenberg_exchange(scan, shells) # fit E(q) for the exchange constants
```

`heisenberg_exchange` fits  $E(\mathbf{q}) - E(0) = m^2 \sum_{\mathbf{R}} J(\mathbf{R}) [1 - \cos(\mathbf{q} \cdot \mathbf{R})]$  over a set of lattice shells, which is how a spiral scan becomes a spin model. `compute_spiral_gradient` gives  $dE/d\mathbf{q}$  on its own.

Also here: **per-atom magnetic fields**, the **topological invariants** of the previous section, and **rank- $n$  tensor symmetrisation** — `symme.f90`'s `symmatrix3/symtensor3` written at any rank.

**Refuses.** A spiral with spin-orbit coupling, permanently — it breaks the theorem, and Elk refuses it too; a spiral with symmetry, until the spin space group is written, so a spiral needs `nosym`; a spiral with ultrasoft/PAW; and spiral relaxation under a magnetic field, whose energy is outside the reported total, so the state is stationary for a different functional than the one being differentiated.

## 10 Performance: dials, memory and GPUs

**Two batching dials** control how much is in flight, and both default to QE's loop — one  $k$ -point and one band at a time, which is what a *cache* wants:

```
PYPRESSO_K_BATCH=all PYPRESSO_BAND_BATCH=all python3 run.py
```

or per call, `run_scf(..., k_batch=...)`. The chunk size changes the order contributions are summed in and nothing else — round-off,  $\sim 1\text{e-}15$  Ry.

**On a GPU both defaults are wrong.** A GPU has no cache to fit in and inverts the conclusion: on a ten-atom metal, `k=1, b=1` costs **4.5x** what `k=all, b=all` does on the same card. Set both. The plausible half-measure `k=all, b=1` is the *worst* setting available, because it buys the batched mode's memory with the looped mode's kernel launches.

**Measured GPU speedups** (one H200 against one CPU core, per SCF iteration): 2x on a cheap metal, 3–5x on norm-conserving silicon, 9–21x with an augmentation charge or a magnetization, and **83–115x on spin-orbit**. Energies agree to  $\sim 1\text{e-}9$  Ry — the same agreements the CPU already has.

**Memory.** Spin-orbit is where it bites: a 20-atom bismuth cell with a relativistic ultrasoft dataset peaks at **34.7 GB**, which fits a 141 GB card and does not fit a 32 GB one. Everything else in that sweep sits under 2 GB.

**Precision.** Every dtype comes from `config.Precision`; `x64` is enabled before any array exists, and all validation is float64. `float32` is a performance mode and **never one a**

**correctness claim is made in.**

## 11 Accuracy summary

Against Quantum ESPRESSO on the same input:

| quantity                                | agreement                                 |
|-----------------------------------------|-------------------------------------------|
| total energy, norm-conserving LDA       | $\sim 1\text{e-}9$ Ry                     |
| ultrasoft, PAW                          | $\leq 3\text{e-}9$ Ry                     |
| PBE / revPBE / PBEsol                   | $\leq 6\text{e-}9$ Ry                     |
| metals, every smearing and tetrahedra   | $2.5\text{e-}8$ Ry                        |
| collinear spin                          | $1.2\text{e-}9$ Ry                        |
| noncollinear magnetism                  | $2.8\text{e-}9$ Ry                        |
| spin-orbit coupling                     | $\leq 1.3\text{e-}8$ Ry                   |
| DFT+U                                   | $\leq 6.7\text{e-}9$ Ry                   |
| band structure                          | 0.0002 eV                                 |
| forces                                  | $\leq 2\text{e-}5$ Ry/bohr                |
| stress (13 cases)                       | $\leq 2.7\text{e-}7$ Ry/bohr <sup>3</sup> |
| relaxed geometry                        | $1\text{e-}6$ bohr                        |
| dielectric constant (NC, US, PAW)       | $\leq 1.2\text{e-}4$                      |
| Born effective charges (NC)             | every printed digit                       |
| phonons at $\Gamma$ , silicon           | $0.05\text{ cm}^{-1}$                     |
| phonons at $\Gamma$ , aluminium (metal) | $0.002\text{ cm}^{-1}$                    |
| Raman/IR spectra vs dynmat.x            | every printed digit                       |

**One case does not close, and is recorded rather than explained.** bi10-soc — ten bismuth atoms with a fully-relativistic dataset — sits **1.9e-4 Ry** from QE on a total of  $-1477.737$ , which is  $1.3\text{e-}7$  relative. Both codes choose the same symmetry operations, the same  $k$ -point, the same grid and the same  $G$ -vectors, and both converge. It is in the test suite at that tolerance, not hidden.

## Where to go next

- README.md — the short tour and a first calculation
- notebooks/ — 26 worked examples on concrete systems, executed and committed, each with a .md export beside it
- PERFORMANCE.md — the running performance log, CPU and GPU
- PLAN.md — the architecture, the phase breakdown, the trap catalogue
- GPU.md — the GPU roadmap